Spaltenorientierte Datenbanken

Pia Georgiew

Gliederung

1. Einführung
2. Kontext
3. Eigenheiten von spaltenorientierten DB
4. Vergleich verschiedener DB
5. Recap
6. Anwendungsbeispiel mit Übung

[1]

Einführung

- Speicherung findet spaltenorientiert statt
- Alle Attribute eines Datentupels liegen am selben Indize
- Ziel: weniger I/O-Operationen beim Ein-/Auslesen der benötigten Daten

ID Number	Last Name	First Name	Bonus
534782	Miller	Ginny	6000
585523	Parker	Peter	8000
479148	Stacy	Gwen	2000

In a Columnar DBMS, the data stored is in this format:

```
534782, 585523, 479148; Miller, Parker, Stacy; Ginny,
Peter, Gwen; 6000, 8000, 2000.
```

In a Row-oriented DBMS, the data stored is in this format:

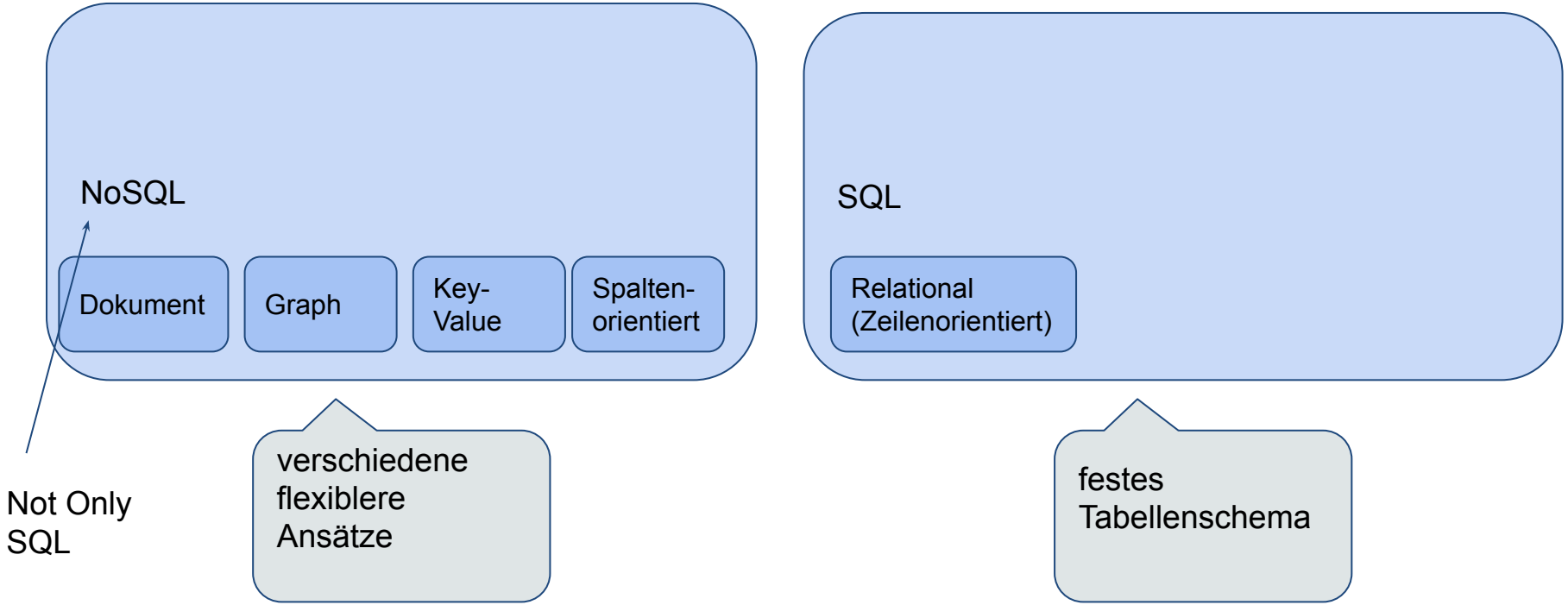
```
534782, Miller, Ginny, 6000; 585523, Parker, Peter,
8000; 479148, Stacy, Gwen, 2000.
```

Kontext: OLTP und OLAP

- „Online Transactional Processing“
- Echtzeit-Transaktionen
- viele verschiedenen Nutzer
- Abfrage/Schreiben von wenigen Einträgen(Tupeln) mit vielen Attributen
- häufig relationale Datenbanken, also einem zeilenorientierten Ansatz

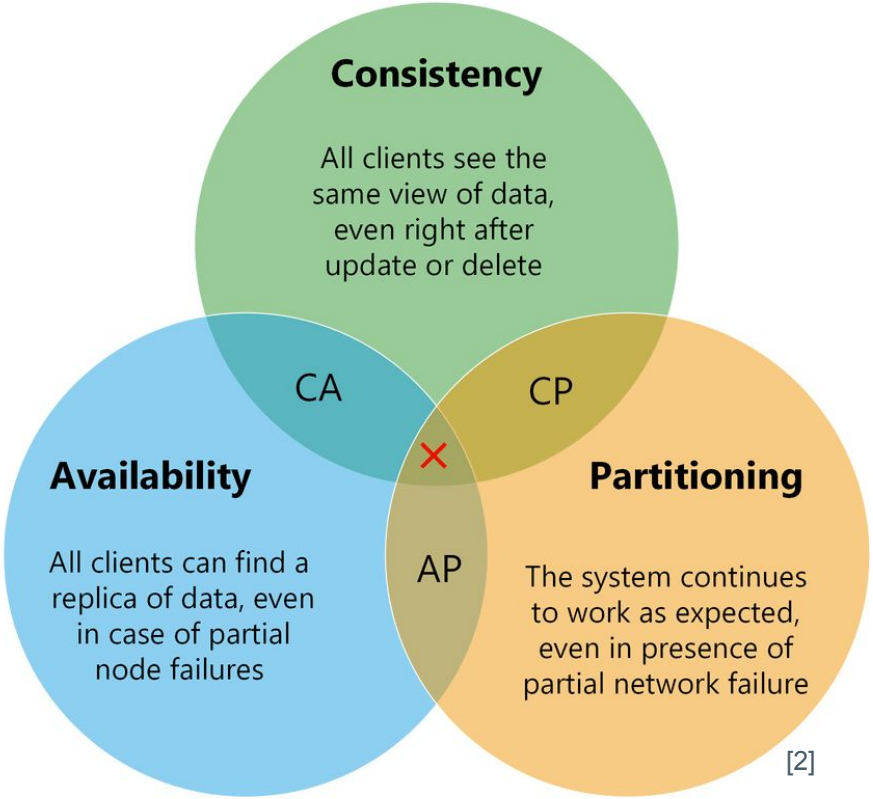
- „Online Analytical Processing“
- analytische Anwendungszwecke
- Entscheidungsunterstützung
- Theorien zur Datendomäne anhand der vorliegenden Daten überprüfen
- häufiges Lesen von großen Datenmengen
- OLAP-Würfel
  - mutidimensionale, datenpunktorientierte Logik
- spaltenorientierte DB

Kontext: NoSQL und SQL



Kontext: CAP-Theorem

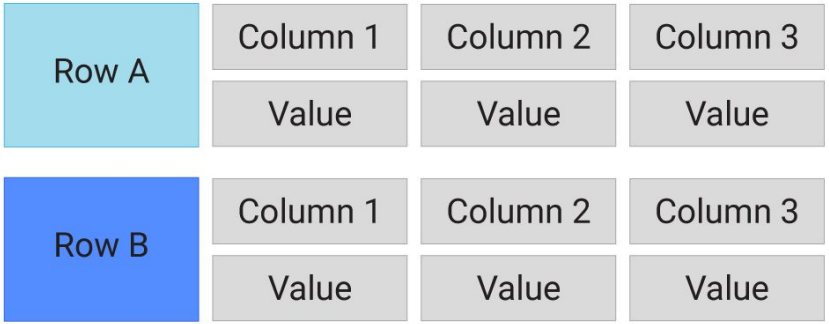
- in verteilten Systemen muss es einen Tradeoff geben - Nur 2 von 3 Eigenschaften können gewährleistet werden
- Partitionierung muss immer gewährleistet werden
- Consistency: Im Falle eines Netzwerkfehlers gibt das System keine Antwort
- Availability: Im Falle eines Netzwerkfehler kann es vorkommen, dass Antworten nicht auf den vollständigen Daten basieren



[2]

Kontext: Wide Column Stores

- Unterscheiden sich stark von spaltenorientierten Datenbanken
- Ähneln eher Key-Value Datenbanken
- Zeilen mit flexibler Anzahl Spalten, durch Schlüssel identifiziert
- Column Family = {Zeilen}
- Keyspace = {Column Family}
- Datenbank = {Keyspaces, System Keyspace}



[3]

Gliederung

1. Einführung
2. Kontext
3. **Eigenheiten von spaltenorientierten DB**
4. Vergleich verschiedener DB
5. Recap
6. Anwendungsbeispiel und Übung

[1]

Speicher- und Einlesestrukturen

- Ausgelegt auf die Abfrage von wenigen Attributen von vielen Datentupeln
- alle Einträge einer Spalte liegen zusammen und können mit wenigen I/O-Operationen eingelesen werden
- Ablage der Spalten in Datenblöcken (2KB - 1MB Größe)
- Append Only Style: statt Zeilen zu aktualisieren, werden häufig neue Einträge mit aktualisierter Gültigkeit angefügt

Speicher- und Einlesestrukturen: Kompression

- Kompression einfacher möglich, da alle Einträge denselben Datentyp haben
- Lauflängencodierung: Anzahl von aufeinanderfolgenden gleichen Werten wird gespeichert
  - z.B. 0000 0111 → 5 3
- Einsatz von Kürzeln und Wörterbüchern
- usw.

Speicher- und Einlesestrukturen: Arbeitsspeicher

- Datenmenge im Arbeitsspeicher soll so klein wie möglich gehalten werden
- Dekompression erfolgt erst während der Berechnung, nicht schon beim Laden
- Abfrage nur von tatsächlich benötigten Spalten
  - z.B. Tabelle mit 100 Spalten; Abfrage benötigt nur 5 Spalten
    - 5 % der Tabelle werden gelesen
    - zeilenorientiert: 100% werden gelesen
  - Effizienzgewinne werden bei sehr hoher Anzahl an Spalten und Zeilen proportional größer

Parallelisierung

- Tabelle wird in Segmente geteilt und auf die Compute-Nodes aufgeteilt
- Jeder Knoten bearbeitet dieselbe Query
- Ergebnisse werden zurückgegeben und vereinigt
- viele unterschiedliche Konzepte dies umzusetzen
  - Redshift: Distribution-, Sortkeys
  - Cassandra/HBase: Map-Reduce
  - …

Parallelisierung: Distribution- und Sortkeys

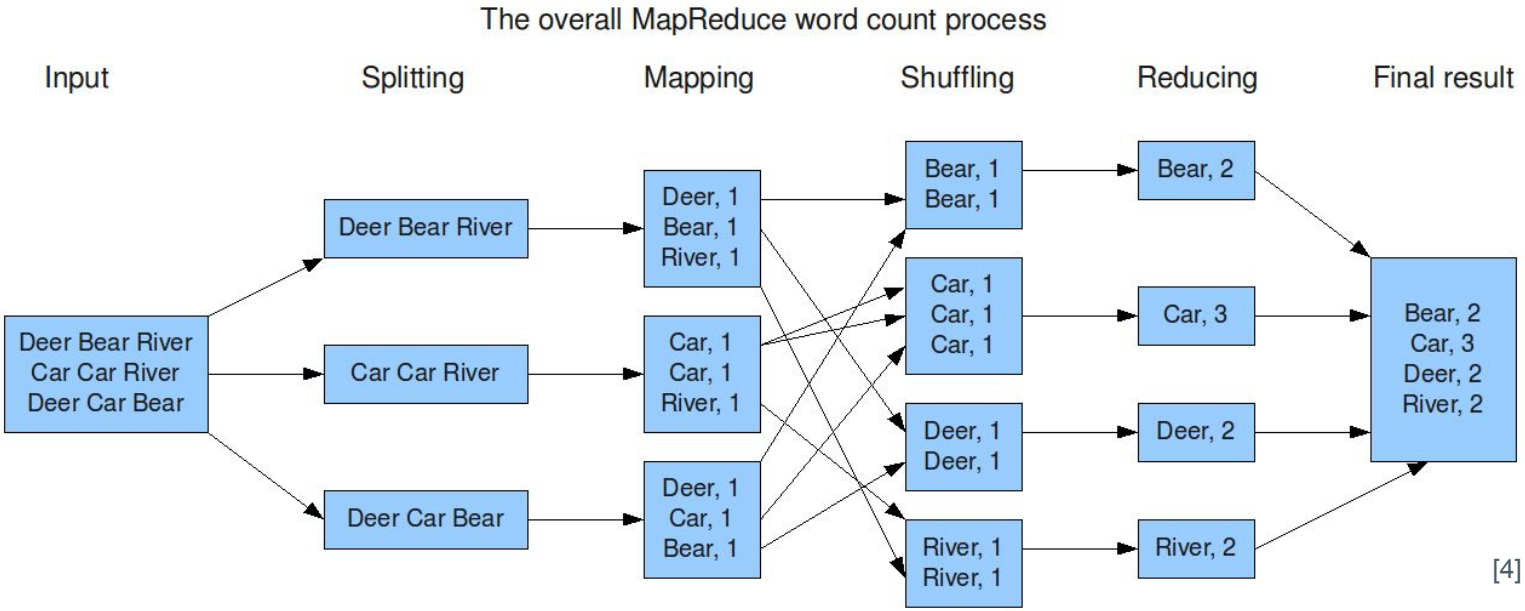
Distributionkeys

- Legt Logik fest nach der die Daten auf die Nodes aufgeteilt werden
  → Nachladen vermeiden
- Alle benötigten Daten zur Querybearbeitung liegen auf Node = Optimale Verteilung
- Rechenlast gleichmäßig verteilen
- bei häufigen Joins
  →Join-Schlüssel = Distkey
- bei Aggregierungen
  → zu aggregierende Spalte = Distkey

Sortkeys

- Legt Sortierung der Tabelle fest
- bestimmte Datenblöcke können übersprungen werden (Min/Max von jedem Datenblock steht zur Verfügung)
- viel Filtern
  →häufigste zu filternde Spalte = Sortkey
- häufigster Zugriff auf neueste Daten
  →Zeitstempelspalte = Sortkey
- häufige Joins
  →Join-Schlüssel = Sortkey

Parallelisierung: Map-Reduce



[4]

Vorteile

- Performant bei OLAP-Anwendungen
  - große Datenmengen
  - Parallelisierung
  - Fensterfunktionen (Min, Max, Average, Rankings, Summen)
- nur tatsächlich benötigte Spalten werden eingelesen
- Effiziente Datenkompression
- Schnelles Lesen

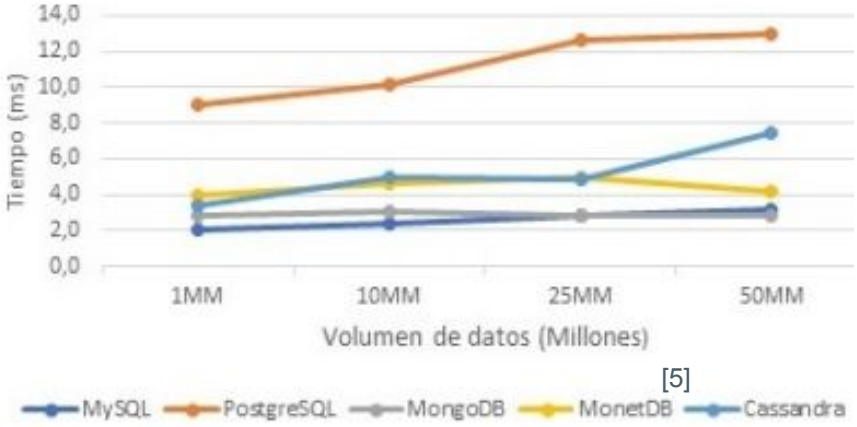
Nachteile

- Langsam beim Einfügen und Bearbeiten von Einträgen
- Langsam bei OLTP Anwendungen
  - Kundenorientiert
  - viele Transaktionen
- Langsam bei häufigen JOIN-Operationen

Performancetest

- Verschiedene Datenbanken sind für verschiedene Abfragen und Anwendungsfälle optimiert
- Vergleich dieser im Paper: “Performance of columnar database” mittels 3 charakteristischer Querys

```
SELECT id, item_nbr, store_nbr, date
FROM train
WHERE id = 500023352;
```



[5]

Figura 7. Results first query

Performancetest

-- Returns 30% of the total data
SELECT id, item_nbr, store_nbr, date
FROM train
WHERE date >= ’2015-12-31’;

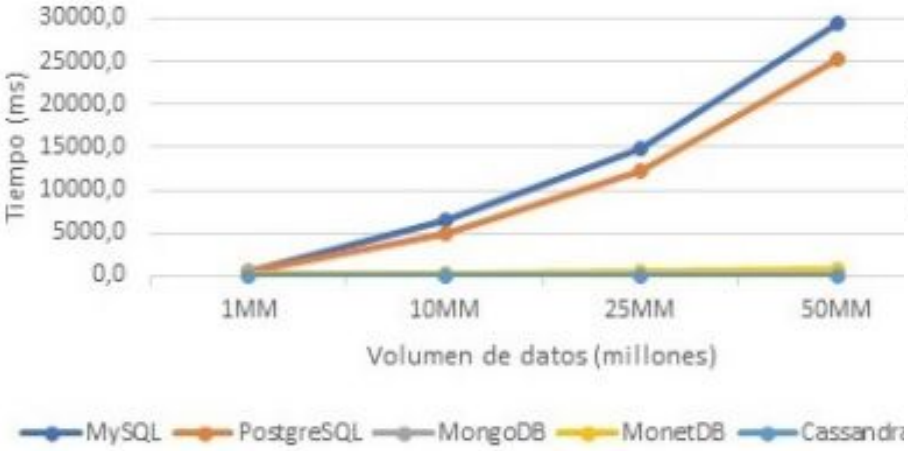


Figura 8. Results second query [5]

SELECT SUM (unit_sales)
FROM train
WHERE store_nbr = ‘12’

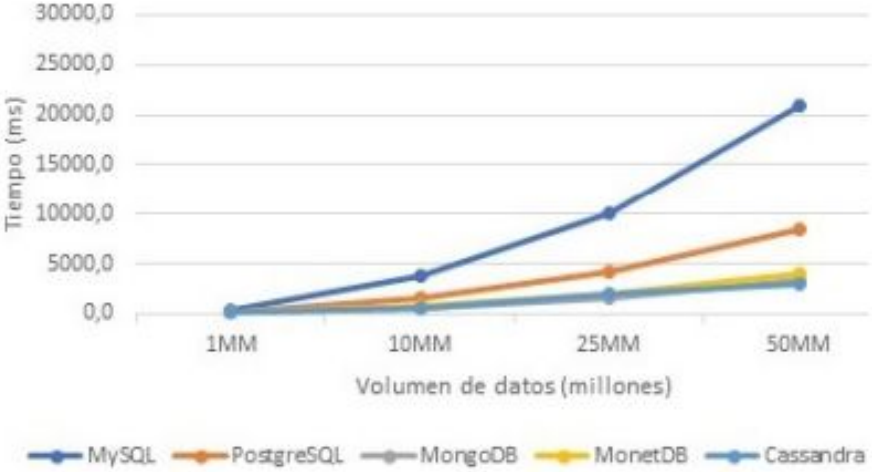


Figura 9. Results third query [5]

Gliederung

1. Einführung
2. Kontext
3. Eigenheiten von spaltenorientierten DB
4. **Vergleich verschiedener DB**
5. Recap
6. Anwendungsbeispiel und Übung

[1]

Vergleich verschiedener Datenbanken

TABLE 2 COLUMN ORIENTED TYPES AND CHARACTERISTICS [11][14]

	PERSISTENCE	REPLICATION	SHARDING	CONSISTENCY	API	QUERY METHOD	IMPLEMENTATION LANGUAGE	CAP
HBASE	built on top of HDFS	Handled on HDFS	By key value, it allows auto sharding	High consistency	Java REST-ful Thrift	MapReduce	Java	CP
HYPERTABLE								CP
CASSANDRA		with zero loss	hashing	consistent	Thrift			AP
ACCUMULO	HDFS	Multi master replication	By row	Eventually consistent	Java REST-ful Thrift	MapReduce	Java	CP

Paper von Tariq N. Khasawneh, Mahmoud H. AL-Sahlee und Ali A. Safia
SQL, NewSQL, and NOSQL Databases: A Comparative Survey
Vergleich von Open-Source verfügbaren spaltenorientierten DB

[6]

Vergleich verschiedener Datenbanken

TABLE 2 COLUMN ORIENTED TYPES AND CHARACTERISTICS [11][14]

PERSISTENCE	REPLICATION	SHARDING	CONSISTENCY	API	QUERY METHOD	IMPLEMENTATION LANGUAGE	CAP

Verfahren nach dem die Datenverteilung auf die Compute-Knoten abläuft

Garantieangabe zur Datenkonsistenz

Angabe der Benutzerschnittstelle

CAP-Theorem

Vergleich verschiedener Datenbanken

TABLE 2 COLUMN ORIENTED TYPES AND CHARACTERISTICS [11][14]

	PERSISTENCE	REPLICATION	SHARDING	CONSISTENCY	API	QUERY METHOD	IMPLEMENTATION LANGUAGE	CAP
HBASE	built on top of HDFS	Handled on HDFS	By key value, it allows auto sharding	High consistency	Java REST-ful Thrift Avro	MapReduce	Java	CP
HYPERTABLE	HDFS	HDFS	Key-value Allows chunking	High consistency	C++ API Java REST-ful Thrift Avro	HQL (Hypertable query language), MapReduce	C++	CP
CASSANDRA	Proprietary format	Auto partition with zero loss	By row hashing	Eventually consistent	CQL Thrift	MapReduce	Java	AP
ACCUMULO	HDFS	Multi master replication	By row	Eventually consistent	Java REST-ful Thrift	MapReduce	Java	CP

Vergleich verschiedener Datenbanken

	Sharding	Consistency	Query Method	CAP
SAP Hana	Range Partitioning, Hashing	High Consistency	SQL, Calculation-/ Analytic-Views	CP
Redshift	Distribution Keys	Eventually Consistency	SQL	AP

Recap

- Einträge werden Spaltenweise gespeichert
- Ziel: Möglichst wenig I/O-Operationen benötigen und schnelle antworten geben
- optimiert auf große Datenmengen und Fensterfunktionen
- Parallelisierung mit verschiedensten Verteilmechnanismen wie Map-Reduce
- Performant bei OLAP-Anwendungen
- Beispiele: SAP Hana, Redshift, MariaDB ColumnStore, HBase, Cassandra
- DB sind sehr unterschiedlich und müssen anwendungsbezogen ausgewählt werden

Gliederung

1. Einführung
2. Kontext
3. Eigenheiten von spaltenorientierten DB
4. Vergleich verschiedener DB
5. Recap
6. **Anwendungsbeispiel und Übung**

[1]

Ticket: Berechnung des Verfügbarkeitsindex

Um die Verfügbarkeit öffentlicher Verkehrsmittel relativ zum Wohnort unserer Kunden (45 Mio) bewerten zu können

möchte ich als Leiter Data Science

die Daten aus dem GTFS Sollfahrplan so aufbereitet haben, dass für jeden Kunden berechnet werden kann, wie viele Abfahrten an den Haltestellen in einem Umkreis von 500m um den Wohnort (angegebene Adresse) herum täglich stattfinden. Da es teils große Unterschiede zwischen den Abfahrten an Werktagen (Mo-Fr) und an den Wochenenden gibt, soll diese Information auch separat für die Tage (Mo-Fr) , Sa und Sonntags ausgewiesen werden.

Darüber hinaus soll die Anzahl der Abfahrten nach NV, RV und FV ausgewiesen werden. Für den RV und FV sollen dabei auch größere Radien herangezogen werden (1000 m, nächster FV-Bahnhof).

Aus den so berechneten Kennzahlen soll ein Index berechnet werden, der die Qualität des Angebots in Relation zum besten Angebot als Index abbildet. Andere Möglichkeiten der sinnvollen Indexierung sind zu prüfen.

GTFS Daten

https://gtfs.de
https://developers.google.com/transit/gtfs/reference?hl=de

GTFS für Deutschland

Im Zuge der Delegierten Verordnung 2017/1926 der Europäischen Union sind seit dem 1.1.2020 die kompletten nationalen Fahrplansolldaten Deutschlands über den Nationalen Zugangspunkt für ÖV-Daten im NeTEx-Format veröffentlicht. Auf Basis dieses Datensatzes bieten wir tagesaktuell generierte Fahrplanddatensätze im GTFS-Format (General Transit Feed Specification) an - für den **kompletten Fern- und Regionalverkehr der Deutschen Bahn sowie den Nahverkehr sämtlicher Verbünde und Anbieter in Deutschland**.

Es handelt sich mit über 20.000 Linien, mehr als 500.000 Haltepunkten und fast 2 Millionen regelmäßig verkehrender Fahrten um **einen der größten GTFS-Datensätze der Welt.**

Zu den Feeds »

[8]

On our way to “Verfügbarkeitsindex”

1. **Welche Tabellen brauchen wir, um für eine Adresse im Umkreis von 1000m alle Haltestellen finden?**
   Und welche um zu wissen, wie viele Abfahrten unter der Woche und am Wochenende stattfinden?

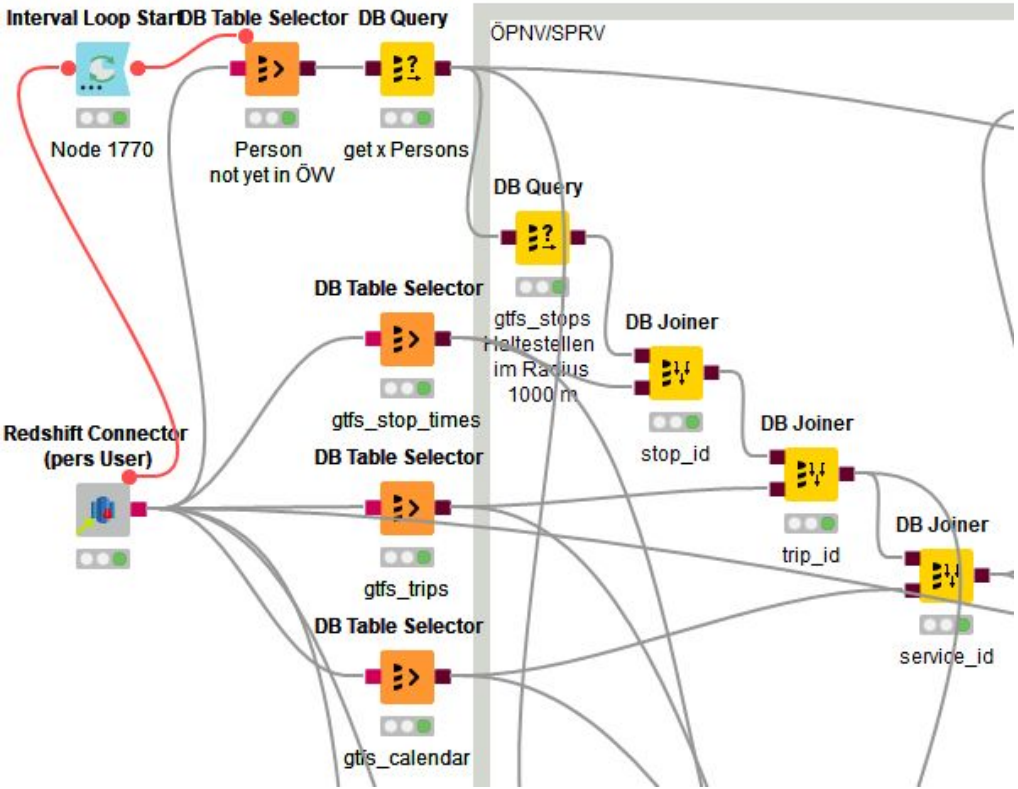
2. Welche Operationen werden wir besonders häufig ausführen?

3. Welche Datenbank sollten wir verwenden?
   Speicherart, Performance, Mögliche Operationen, CAP-Theorem, Persönliche Präferenz,..

4. Auf was muss bei der Integration der Tabellen geachtet werden? Welche Schritte werden beim Erstellen einer Tabelle ausgeführt?

Dateiname	Erforderlich	Definiert
`agency.txt`	**Erforderlich**	Betreiber mit in diesem Dataset dargestelltem Fahrbetrieb.
`stops.txt`	**Erforderlich**	Haltestellen, an denen Fahrgäste ein- oder aussteigen können. Definiert auch Stationen und Stationseingänge.
`routes.txt`	**Erforderlich**	Routen mit öffentlichen Verkehrsmitteln. Eine Route ist eine Gruppe von Fahrten, die Fahrgästen als eine einzelne Strecke angezeigt werden.
`trips.txt`	**Erforderlich**	Fahrten auf der jeweiligen Route. Eine Fahrt ist eine Abfolge von zwei oder mehr Haltestellen innerhalb einer bestimmten Zeit.
`stop_times.txt`	**Erforderlich**	Ankunfts- und Abfahrtszeiten an Haltestellen für die jeweilige Fahrt.
`calendar.txt`	**Bedingt erforderlich**	Betriebstage, die in einem wöchentlichen Zeitplan mit Start- und Enddatum angegeben werden. Diese Datei ist erforderlich, falls nicht alle Betriebstage in `calendar_dates.txt` festgelegt sind.
`calendar_dates.txt`	**Bedingt erforderlich**	Ausnahmen für die in `calendar.txt` definierten Betriebstage. Wenn `calendar.txt` weggelassen wird, ist die Datei `calendar_dates.txt` erforderlich und muss alle Betriebstage enthalten.

[8]



Columns: 24	DB Type	DB Type Na...	KNIME Type
person_wid	NUMERIC	numeric	D Number (double)
longitude	DOUBLE	float8	D Number (double)
latitude	DOUBLE	float8	D Number (double)
point	LONGVARBI...	geometry	Binary object
kundennummer_kto	VARCHAR	varchar	S String
stop_lon	DOUBLE	float8	D Number (double)
stop_lat	DOUBLE	float8	D Number (double)
stop_name	VARCHAR	varchar	S String
stop_id	INTEGER	int4	I Number (integer)
trip_id	INTEGER	int4	I Number (integer)
arrival_time	VARCHAR	varchar	S String
stop_sequence	INTEGER	int4	I Number (integer)
route_id	INTEGER	int4	I Number (integer)
service_id	INTEGER	int4	I Number (integer)
monday	INTEGER	int4	I Number (integer)
tuesday	INTEGER	int4	I Number (integer)
wednesday	INTEGER	int4	I Number (integer)
thursday	INTEGER	int4	I Number (integer)
friday	INTEGER	int4	I Number (integer)
saturday	INTEGER	int4	I Number (integer)
sunday	INTEGER	int4	I Number (integer)
start_date	DATE	date	31 Local Date
end_date	DATE	date	31 Local Date
bucket	VARCHAR	varchar	S String

stop_name	stop_id	trip_id	arrival_...	stop_se...	route_id	service_id	monday	tuesday	wednes...	thursday	friday	saturday	sunday
Hannover Thielenplatz/Schausp.	1502	11102784	22:07	18	113148	1178	1	1	1	1	1	0	0
Hannover Thielenplatz/Schausp.	1502	11192162	09:59	18	113148	1178	1	1	1	1	1	0	0
Hannover Thielenplatz/Schausp.	1502	1127721	19:22	18	113148	12401	0	0	0	0	0	0	1
Hannover Thielenplatz/Schausp.	1502	11282660	22:52	18	113148	1178	1	1	1	1	1	0	0

Zu jedem Stop haben wir alle Trips (also Fahrten). Jeder Trip hat eine Service ID, welche angibt, an welchen Tagen dieser Trip stattfindet.

On our way to “Verfügbarkeitsindex”

1. Wie können wir für einen Kunden im Umkreis von 1000m um seine Adresse alle Haltestellen finden? Und wie viele Abfahrten finden dort unter der Woche und am Wochenende statt?

2. **Welche Operationen werden wir besonders häufig ausführen?**

3. **Welche Datenbank sollten wir verwenden?**
   Speicherart, Performance, Mögliche Operationen, CAP-Theorem, Persönliche Präferenz,..

4. Auf was muss bei der Integration der Tabellen geachtet werden? Welche Schritte werden beim Erstellen einer Tabelle ausgeführt?

On our way to “Verfügbarkeitsindex”

1. Wie können wir für einen Kunden im Umkreis von 1000m um seine Adresse alle Haltestellen finden? Und wie viele Abfahrten finden dort unter der Woche und am Wochenende statt?

2. Welche Operationen werden wir besonders häufig ausführen?

3. Welche Datenbank sollten wir verwenden?
   Speicherart, Performance, Mögliche Operationen, CAP-Theorem, persönliche Präferenz,..

4. **Auf was muss bei der Integration der Tabellen in die Datenbank geachtet werden?**
   Welche Schritte werden beim Erstellen einer Tabelle ausgeführt?
   Können wir die Performance noch verbessern?

Performance

```
EXPLAIN
SELECT sum(monday), sum(tuesday), sum(wednesday), sum(thursday), sum(friday), sum(saturday), sum(sunday)
FROM
  (SELECT *
   FROM
     (SELECT person_wid, trip_id, NearByServices.service_id, monday, tuesday, wednesday, thursday, friday, saturday, sunday
      FROM
        (SELECT person_wid, POINT, stop_name, stop_id, NearByTrips.trip_id, playground.pg_gtfs_trips.route_id, playground.pg_gtfs_trips.service_id
         FROM
           (SELECT person_wid, POINT, stop_name, parent_station, NearbyStops.stop_id, trip_id
            FROM (
                    (SELECT *
                     FROM refined.geo_koordinaten_kundenadressen gkk
                     WHERE person_wid = 37503398)
                   JOIN playground.pg_gtfs_stops AS stops ON ST_DWithin("point", ST_SetSRID(ST_MakePoint(stops.stop_lon, stops.stop_lat), 4326), 1/110.57)) AS NearbyS
            LEFT JOIN playground.pg_gtfs_stop_times ON NearbyStops.stop_id = playground.pg_gtfs_stop_times.stop_id) AS NearByTrips
         LEFT JOIN playground.pg_gtfs_trips ON NearbyTrips.trip_id = playground.pg_gtfs_trips.trip_id) AS NearByServices
      LEFT JOIN playground.pg_gtfs_calendar ON NearbyServices.service_id = playground.pg_gtfs_calendar.service_id)
  GROUP BY "person_wid", trip_id, service_id, monday, tuesday, wednesday, thursday, friday, saturday, sunday)
```

QUERY PLAN

```
XN Aggregate  (cost=1101611042530.60..1101611042530.60 rows=1 width=28)
 -> XN Subquery Scan derived_table3  (cost=1101611033559.39..1101611036821.64 rows=326225 width=28)
    -> XN HashAggregate  (cost=1101611033559.39..1101611033559.39 rows=326225 width=52)
       -> XN Hash Left Join DS_BCAST_INNER  (cost=1030597747415.18..1101611025403.77 rows=326225 width=52)
          Hash Cond: ("outer".service_id = "inner".service_id)
          -> XN Hash Right Join DS_DIST_BOTH  (cost=1030597747340.61..1099224618114.27 rows=326225 width=24)
             Outer Dist Key: gtfs_trips.trip_id
             Inner Dist Key: gtfs_stop_times.trip_id
             Hash Cond: ("outer".trip_id = "inner".trip_id)
             -> XN Seq Scan on gtfs_trips  (cost=0.00..15782.85 rows=1578285 width=8)
             -> XN Hash  (cost=1030597746531.60..1030597746531.60 rows=323602 width=20)
                -> XN Hash Right Join DS_DIST_BOTH  (cost=1162925.23..1030597746531.60 rows=323602 width=20)
                   Outer Dist Key: gtfs_stop_times.stop_id
                   Inner Dist Key: stops.stop_id
                   Hash Cond: ("outer".stop_id = "inner".stop_id)
                   -> XN Seq Scan on gtfs_stop_times  (cost=0.00..321942.38 rows=32194238 width=8)
                   -> XN Hash  (cost=1162908.25..1162908.25 rows=6790 width=20)
                      -> XN Nested Loop DS_BCAST_INNER  (cost=0.00..1162908.25 rows=6790 width=20)
                         Join Filter: (true = st_dwithin("inner".point, st_setsrid(st_makepoint("outer".stop_lon, "outer".stop_lat), 4326), 0.009044044496698
                         -> XN Seq Scan on gtfs_stops stops  (cost=0.00..6789.45 rows=678945 width=20)
                         -> XN Seq Scan on geo_koordinaten_kundenadressen gkk  (cost=0.00..0.01 rows=2 width=48)
                            Filter: (person_wid = 37503398::numeric)
          -> XN Hash  (cost=59.66..59.66 rows=5966 width=32)
             -> XN Seq Scan on gtfs_calendar  (cost=0.00..59.66 rows=5966 width=32)
----- Nested Loop Join in the query plan - review the join predicates to avoid Cartesian products -----
```

QUERY PLAN

```
XN Aggregate  (cost=20509744546.67..20509744546.67 rows=1 width=28)
  -> XN Subquery Scan derived_table3  (cost=20509735575.46..20509738837.71 rows=326225 width=28)
    -> XN HashAggregate  (cost=20509735575.46..20509735575.46 rows=326225 width=52)
      -> XN Hash Left Join DS_BCAST_INNER  (cost=1569431.25..20509727419.84 rows=326225 width=52)
        Hash Cond: ("outer".service_id = "inner".service_id)
        -> XN Hash Right Join DS_DIST_INNER  (cost=1569356.68..18123320130.34 rows=326225 width=24)
          Inner Dist Key: pg_gtfs_stop_times.trip_id
          Hash Cond: ("outer".trip_id = "inner".trip_id)
          -> XN Seq Scan on pg_gtfs_trips  (cost=0.00..15782.85 rows=1578285 width=8)
          -> XN Hash  (cost=1568547.67..1568547.67 rows=323602 width=20)
            -> XN Merge Left Join DS_DIST_NONE  (cost=0.00..1568547.67 rows=323602 width=20)
              Merge Cond: ("outer".stop_id = "inner".stop_id)
              -> XN Nested Loop DS_BCAST_INNER  (cost=0.00..1162908.25 rows=6790 width=20)
                Join Filter: (true = st_dwithin("inner".point, st_setsrid(st_makepoint("outer".stop_lon, "outer".stop_lat), 4326), 0.0090440444966989001::double precision))
                -> XN Seq Scan on pg_gtfs_stops stops  (cost=0.00..6789.45 rows=678945 width=20)
                -> XN Seq Scan on geo_koordinaten_kundenadressen gkk  (cost=0.00..0.01 rows=2 width=48)
                  Filter: (person_wid = 37503398::numeric)
              -> XN Seq Scan on pg_gtfs_stop_times  (cost=0.00..321942.36 rows=32194236 width=8)
        -> XN Hash  (cost=59.66..59.66 rows=5966 width=32)
          -> XN Seq Scan on pg_gtfs_calendar  (cost=0.00..59.66 rows=5966 width=32)
----- Nested Loop Join in the query plan - review the join predicates to avoid Cartesian products -----
```

GTFS Tabelle einspielen in Cassandra

1. Docker installieren
https://docs.docker.com/get-docker/

2. Docker Image Cassandra installieren und Volume angeben
https://cassandra.apache.org/doc/latest/cassandra/getting_started/installing.html#installing-the-docker-image
https://docs.docker.com/storage/volumes/

```
docker pull cassandra:latest
docker run --name cass_cluster -v $(pwd)/Dokumente/DB2/latest:/gtfs
    cassandra:latest

// docker ps -a
// docker rm cass_cluster
```

3. Cqlsh Konsole Öffnen
```
docker exec -it cass_cluster cqlsh
```

4. Keyspace und Tabelle erstellen & danach die GTFS Tabelle über den Befehl COPY einlesen
```
CREATE KEYSPACE meinkeyspace
  WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
  };

CREATE TABLE meinkeyspace.agency (
  agency_id int PRIMARY KEY,
  agency_name text,
  agency_url text,
  agency_timezone text,
  agency_lang text
);

COPY meinkeyspace.agency (agency_id, agency_name, agency_url,
    agency_timezone, agency_lang) FROM 'gtfs/agency.csv' WITH HEADER = true;
```

5. Austoben ;)

GTFS Tabelle einspielen in MariaDB Column Store

1. Docker installieren
https://docs.docker.com/get-docker/

2. Docker Image MariaDB Column Store installieren und Volume angeben
https://hub.docker.com/r/mariadb/columnstore
https://docs.docker.com/storage/volumes/

```
docker pull mariadb/columnstore
docker run -d -p 3307:3306 --shm-size=512m -e PM1=mcs1 -v
   $(pwd)/Dokumente/DB2/gtfs:/gtfs --hostname=mcs1 --name mcs1
   mariadb/columnstore
docker exec -it mcs1 provision mcs1
docker ps
```

3. Mysql Konsole Öffnen
mysql -h 127.0.0.1 -P 3307 -u admin -p
Password: C0lumnStore!

4. Datenbank und Tabelle erstellen & danach die GTFS Tabelle über den Befehl LOAD DATA INFILE einlesen

```
Create Database gtfs;
CREATE TABLE gtfs.agency (   agency_id int PRIMARY KEY,
   agency_name text,   agency_url text,   agency_timezone text,
   agency_lang text );
LOAD DATA INFILE '/gtfs/agency.csv' INTO TABLE gtfs.agency FIELDS
   TERMINATED BY ',' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
CREATE TABLE gtfs.stops (   stop_name VARCHAR(255),
   parent_station VARCHAR(255),   stop_id INT,   stop_lat
   DECIMAL(12,8), stop_lon DECIMAL(12,8), location_type
   VARCHAR(3));
LOAD DATA INFILE '/gtfs/stops.csv' INTO TABLE gtfs.stops FIELDS
   TERMINATED BY ';' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
```

5. Austoben ;)

Bildquellen

[1] https://tracking-garden.com/site/assets/files/1336/data.800x0.png
[2] https://thecustomizewindows.com/2021/01/cap-theorem-explained/
[3]https://www.scylladb.com/wp-content/uploads/Wide-column-Database-diagram.png
[4]https://wi-wiki.de/doku.php?id=bigdata:mapreduce
[5] https://ingenius.ups.edu.ec/index.php/ingenius/article/view/22.2019.05/3280
[6] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9078970
[7] http://gtfs.de/